Design and Implementation of a 1/10 Scale Autonomous Vehicle Using ROS2 and raspberry pi

Moustafa Ahmed
ID: 55-12575
Moustafa.abuelmagd@student.guc.edu.eg

Hazim Ashraf
ID: 55-12843

Ahmed Mostafa
ID: 55-13264

Abdallah Hatem
ID: 55-12791

***Abstract*—This paper presents the development and implementation of a 1/10 scale autonomous vehicle using Raspberry Pi and Arduino integrated with ROS2 Jazzy and Gazebo simulation. The project focused on autonomous navigation, localization, and control using sensor fusion techniques. An Extended Kalman Filter (EKF) was implemented for vehicle localization using IMU and encoder data. PID control was used for longitudinal speed control, while the Stanley Controller algorithm was implemented for lateral trajectory tracking. The system was tested in Gazebo simulation on multiple tracks including obstacle avoidance and closed-loop trajectory tracking. The report presents the complete workflow from system design and hardware integration to localization and control implementation.**

***Index Terms*—Autonomous Vehicle, ROS2 Jazzy, Gazebo, EKF, Stanley Controller, PID Control, Raspberry Pi, Arduino, Sensor Fusion**

I. Introduction

Autonomous vehicles represent one of the fastest-growing research fields in robotics and intelligent transportation systems. Modern autonomous systems require the integration of localization, perception, control, and planning algorithms into a complete robotic platform capable of operating without human intervention.

The goal of this project was to develop a 1/10 scale autonomous vehicle capable of following predefined trajectories and navigating through different driving scenarios using ROS2 Jazzy and Gazebo simulation.

The vehicle was developed using a Raspberry Pi as the main processing unit and an Arduino microcontroller for low-level motor interfacing and sensor acquisition. The system used an IMU sensor and wheel encoders for localization and motion estimation.

The project objectives included:

- Developing a complete autonomous vehicle architecture.
- Implementing localization using an Extended Kalman Filter.
- Designing longitudinal and lateral controllers.
- Testing the vehicle in Gazebo simulation.
- Evaluating obstacle avoidance and track following performance.

II. System Architecture

The autonomous vehicle system consisted of both hardware and software components integrated together to achieve autonomous driving functionality.

A. Hardware Components

The hardware architecture included:

- Raspberry Pi as the main onboard computer.
- Arduino microcontroller for low-level control.
- DC motor and motor driver.
- Steering servo motor.
- Inertial Measurement Unit (IMU).
- Wheel encoders.
- Battery power system.

The Raspberry Pi handled high-level algorithms such as localization, control, and ROS2 communication. The Arduino was responsible for motor control and encoder data acquisition.

B. Software Architecture

ROS2 Jazzy was used as the middleware framework due to its modularity and scalability for robotic systems.

The software architecture consisted of multiple ROS2 nodes including:

- IMU processing node.
- Encoder processing node.
- EKF localization node.
- PID speed controller node.
- Stanley lateral controller node.
- Vehicle kinematics node.
- Gazebo interface node.

ROS2 topics were used for communication between nodes, enabling modular development and easier debugging.

III. Vehicle Modeling and Simulation

Gazebo simulation was used to validate the implemented algorithms before deployment on real hardware.

A. Vehicle Model

A complete vehicle model was developed in Gazebo including:

- Rear-wheel drive system.
- Front steering mechanism.
- IMU sensor plugin.
- Encoder simulation.
- Vehicle kinematic model.

The simulation model was designed to replicate the behavior of the real 1/10 scale autonomous vehicle.

B. ROS2 and Gazebo Integration

ROS2 Jazzy was integrated with Gazebo through ROS2 plugins and topic communication.

The simulation environment enabled:

- Publishing simulated sensor data.
- Sending steering and speed commands.
- Visualizing trajectories in RViz.
- Testing localization and control algorithms.

IV. Localization Using Extended Kalman Filter

Localization is one of the most important components in autonomous driving systems. In this project, an Extended Kalman Filter (EKF) was implemented to estimate vehicle position and orientation.

A. Sensor Fusion

The EKF combined data from:

- IMU sensor measurements.
- Wheel encoder velocity measurements.

The state vector was defined as:

$$x = \begin{bmatrix} x \\ y \\ \theta \end{bmatrix} \tag{1}$$

where:

- x and y represent vehicle position.
- θ represents vehicle orientation.

B. Prediction Model

The vehicle motion model was implemented using the following equations:

$$x_{k+1} = x_k + v\cos(\theta)\Delta t \tag{2}$$

$$y_{k+1} = y_k + v\sin(\theta)\Delta t \tag{3}$$

$$\theta_{k+1} = \theta_k + \omega\Delta t \tag{4}$$

where:

- v is the linear velocity.
- ω is the angular velocity.
- Δt is the sampling time.

The EKF corrected the predicted state using sensor measurements to improve localization accuracy.

C. EKF Trajectory Results

The EKF localization was tested on different tracks inside the Gazebo simulation environment.

1) Track 2: Obstacle Avoidance: Track 2 focused on avoiding two obstacles while maintaining trajectory tracking accuracy.

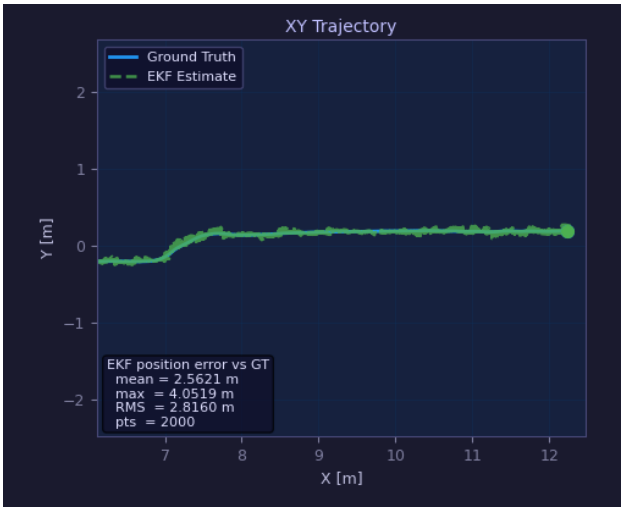

Fig. 1. EKF trajectory estimation for obstacle avoidance track

The EKF successfully estimated the vehicle trajectory while navigating around the obstacles.

2) Track 3: Closed Loop Navigation: Track 3 involved driving around a complete closed-loop track.

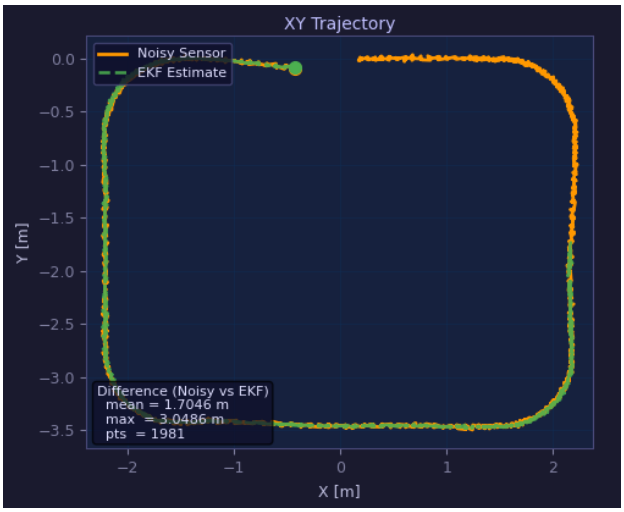

Fig. 2. EKF trajectory estimation for closed-loop track

The localization system maintained stable position estimation throughout the complete trajectory.

V. Control System Design

The vehicle control system consisted of longitudinal speed control and lateral steering control.

A. PID Speed Controller

A PID controller was implemented to regulate vehicle speed.

The PID control law is defined as:

$$u(t) = K_p e(t) + K_i \int e(t)dt + K_d \frac{de(t)}{dt} \tag{5}$$

where:

- K_p is the proportional gain.
- K_i is the integral gain.
- K_d is the derivative gain.
- $e(t)$ is the speed error.

The controller adjusted motor PWM values to maintain the desired reference speed while minimizing overshoot and steady-state error.

B. Stanley Controller for Lateral Control

The Stanley Controller algorithm was implemented for lateral path tracking.

The steering angle was computed using:

$$\delta = \theta_e + \tan^{-1}\left(\frac{ke}{v}\right) \tag{6}$$

where:

- δ is the steering angle.
- θ_e is the heading error.
- e is the cross-track error.
- v is the vehicle velocity.
- k is the control gain.

The Stanley controller provided smooth steering behavior and minimized trajectory tracking error.

VI. Simulation Results

The developed autonomous vehicle system was tested in Gazebo simulation under different driving scenarios.

A. Obstacle Avoidance Performance

The vehicle successfully navigated around two obstacles while maintaining trajectory tracking performance. The Stanley controller generated stable steering commands and prevented excessive oscillations.

B. Track Following Performance

The vehicle demonstrated reliable closed-loop trajectory tracking with stable speed regulation and accurate steering behavior.

C. Localization Accuracy

The EKF significantly improved localization accuracy compared to raw sensor measurements by reducing noise and drift.

VII. Challenges and Solutions

Several technical challenges were encountered during the project development process.

A. Sensor Noise

Encoder and IMU data contained measurement noise and drift. The EKF sensor fusion algorithm reduced these effects and improved state estimation.

B. Controller Tuning

PID and Stanley controller gains required multiple tuning iterations to achieve stable and smooth vehicle behavior.

C. ROS2 Communication

Synchronization between ROS2 nodes and Gazebo simulation introduced timing challenges. Proper ROS2 topic configuration improved communication reliability.

VIII. Autonomous Vehicle Platform

The final autonomous vehicle integrated all hardware and software components into a complete robotic system.

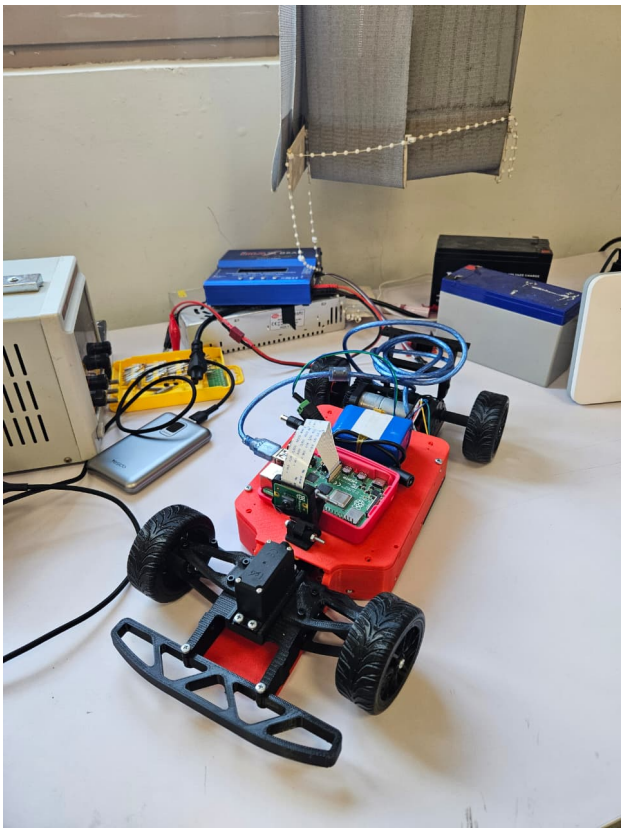

Fig. 3. 1/10 scale autonomous vehicle

The platform demonstrated the successful integration of embedded systems, robotics software, localization algorithms, and control systems.

IX. Conclusion

This project presented the development of a 1/10 scale autonomous vehicle using Raspberry Pi, Arduino, ROS2 Jazzy, and Gazebo simulation.

The implemented system successfully achieved autonomous trajectory tracking through the integration of localization and control algorithms. An Extended Kalman Filter was implemented for sensor fusion using IMU and encoder data. PID control was used for speed regulation, while the Stanley Controller algorithm was used for lateral path tracking.

Simulation testing demonstrated successful obstacle avoidance and closed-loop track following capabilities. The project provided practical experience in robotics, autonomous systems, ROS2 architecture, and vehicle control.

Future work may include:

- Real-world hardware deployment.
- Integration of LiDAR or camera perception systems.
- SLAM implementation.
- Advanced path planning algorithms.
- Model Predictive Control (MPC).

ACKNOWLEDGMENT

The authors would like to thank the project supervisors dr,Eng Dalia and Professor Omar shehata and team members who contributed to the successful completion of this project.

REFERENCES

[1] Open Robotics, "ROS2 Documentation," 2025.
[2] Gazebo Simulator Documentation, Open Robotics, 2025.
[3] S. Thrun et al., "Stanley: The Robot that Won the DARPA Grand Challenge," Journal of Field Robotics, vol. 23, no. 9, pp. 661–692, 2006.
[4] G. Welch and G. Bishop, "An Introduction to the Kalman Filter," University of North Carolina, 2006.
[5] K. Ogata, "Modern Control Engineering," 5th Edition, Prentice Hall, 2010.